

RAG and Downstream Layers

Implemented features, selection rules, similarity calculations, ordering, limits, and end-to-end flow

[Chat With Unknown Data | Code-based reference | August 2026](#)

Scope: What the current code implements in RAG (Layer 3), SQL Generation (Layer 4), Self-Correction (Layer 5), and Answer Generation (Layer 6).

1. Pipeline Overview

Question -> metadata shortcut -> introspection and enrichment -> RAG -> SQL generation -> execution and self-correction -> answer generation

Metadata shortcut

Structural questions can bypass embeddings and SQL. The pipeline directly answers requests to list or count schemas/tables, and can list or count columns when an exact qualified table is supplied.

Normal data-question path

1. Introspection supplies authoritative schemas, tables, columns, types, nullability, primary keys, and foreign keys.
2. Enrichment supplies optional table and column descriptions.
3. RAG selects relevant schema context.
4. The chat model produces T-SQL.
5. Self-correction executes it and treats errors differently from empty successes.
6. Answer generation converts successful rows into a concise response.

Layer	Main input	Main output
RAG	Question plus enriched metadata	Compact relevant schema
SQL generation	Question plus compact schema	T-SQL
Self-correction	SQL, question, schema, database	Rows plus final SQL
Answer generation	Question, final SQL, rows	User-facing answer

2. RAG Documents and Embedding Index

Table documents

Each contains its schema/table identifier, enriched purpose, column names, and enriched column descriptions. Table similarity therefore reflects the table name, purpose, and meanings of its columns.

Column documents

Each contains its owning table, column name, data type, description, primary-key status, and foreign-key target. Ranking can use both business meaning and structural metadata.

Persistent cache

- Vectors are stored under `.cache/embeddings/` with a database-derived filename.
- The embedding model name and a SHA-256 document digest control reuse.
- Only new or changed documents are embedded again; the index is also retained in memory.
- Batches contain at most 32 documents or 24,000 characters.
- Failed batches are retried one document at a time.
- Empty, infinite, and NaN vectors are rejected. Reproducible NaN cases use safe metadata formatting.

Embedding model

The default is local Ollama `bge-m3` for multilingual Persian/English retrieval. The complete question is embedded once and reused for table and column scoring.

3. RAG Table Selection and Order

Step 1: exact qualified-table detection

The question is checked for an exact case-insensitive `schema.table`. Brackets and common quotes are ignored. If found, semantic table selection is bypassed and at most eight matches are kept. Multiple matches follow introspection order, not mention order. The query vector is still used for columns.

Step 2: semantic similarity

Every table receives cosine similarity:

```
table score = cosine(question vector, table-document vector)
```

The initial order is descending cosine score. Only the top 24 candidates reach identifier reranking.

Step 3: compound/base reranking

PascalCase, snake_case, dots, and hyphens are split into lowercase tokens. There is no predefined business-word list. A base is a strict token subset of a compound table in the same schema.

Question	Base	Compound	Result
show contracts	Contract	ContractCoefficient	Prefer base when scores are close
show contract coefficient	Contract	ContractCoefficient	Keep compound

The rule activates only when the compound leads by 0.00 to 0.05 and none of its extra tokens appears in the question. The base receives +0.03 and compound -0.015, followed by another sort. Tokens are exact; singular/plural normalization is not implemented.

Step 4: cutoff

```
threshold = max(0.30, best adjusted score - 0.10)
```

After reranking only eight candidates remain. Tables meeting the threshold are retained in score order. If none qualifies, the closest table is retained even below 0.30.

4. FK Expansion and Column Selection

Direct FK targets

After semantic selection, direct FK-referenced tables are appended. Semantic tables stay first; FK targets follow in selected-table and column-metadata order. Duplicates are removed. Expansion is one hop, does not follow reverse FKs, and may raise the final table count above eight.

```
Order selected -> Order.CustomerId references Customer.Id -> append Customer
```

Column similarity

Columns inside the final table set are ordered using cosine similarity between the question vector and each column document.

Tables with 30 or fewer columns

All columns are included. PK/FK columns are promoted first, followed by other columns. Both groups preserve semantic-score order.

Tables with more than 30 columns

1. Include every PK and FK column.
2. Include identifiers explicitly present in the normalized question.
3. Add highest-scoring columns until at least 20 are selected.
4. Add siblings differing by one prefix or suffix token, such as `Title`/`Title_En`.

Structural and sibling additions can raise the result above 20 columns.

Compact schema

Output includes purpose, name, type, nullability, PK, FK target, and description/comment. A 21,000-character shared limit applies. Columns are allocated round-robin across tables, and omitted counts are shown.

5. RAG Decision Reference

Decision	Implemented method
Explicit table	Exact lexical schema.table match
Initial table order	Cosine similarity descending
Compound/base adjustment	Token-subset rule among top 24
Semantic cutoff	At most 8; score at least max(0.30, best - 0.10)
Low-score fallback	Always retain closest table
Join context	Append direct outgoing FK targets
Column order	Cosine similarity with structural promotion
PK/FK columns	Always retained
Small tables	All columns at 30 or fewer
Wide tables	Keys, explicit names, top 20+, siblings
Prompt size	21,000-character round-robin budget

Similarity and matching types

- **Cosine similarity:** numerical semantic table/column ranking.
- **Exact qualified-name match:** table-selection override.
- **Identifier token subset:** compound/base reranking.
- **Normalized containment:** explicit wide-table column retention.
- **One-token prefix/suffix relation:** sibling column retention.

6. Layer 4 - SQL Generation

One chat-model call uses only the original question and compact RAG schema.

Prompt constraints

- Generate SQL Server T-SQL and output only SQL.
- Start with `SELECT`.
- Use bracketed `[schema].[table]` names and aliases.
- Use `TOP`, not `LIMIT`.
- Use `N'...'` for non-ASCII literals.
- Prefer the simplest directly useful table.
- Avoid joins unless needed and supported by the schema.

Output cleanup

Markdown fences are removed, repeated leading `SELECT SELECT` is collapsed, and content starting at the first `SELECT` or `WITH` is extracted. Output with neither is rejected.

Limitation: This uses regular expressions, not a SQL AST, and does not independently prove that the complete statement is read-only.

7. Layer 5 - Execution and Empty Results

Non-empty success

SQL executes through SQLAlchemy. Returned rows and successful SQL are immediately passed onward without another LLM call.

Empty success

An empty result is handled separately from an error. At most one grounded correction is attempted; the model is not asked to invent values.

1. Parse qualified bracketed tables from FROM/JOIN, aliases, and exact string equalities.
2. Inspect at most three predicates and require a real string column.
3. Probe the exact column with escaped full tokens and three-character prefixes, returning at most 200 distinct values.
4. Keep at most five candidates scoring at least 0.70.
5. Give the model the question, schema, empty SQL, [], and grounded candidates.

Candidate similarity

$0.70 \times \text{whole-string SequenceMatcher} + 0.20 \times \text{token-overlap F1} + 0.10 \times \text{average best-token edit similarity}$

Comparison normalization covers case, punctuation, Persian/Arabic character variants, diacritics, and zero-width non-joiners. Original values are preserved for SQL. Without grounded candidates, the empty result is returned unchanged.

8. Error Correction and Retry Rules

Error context

The model receives the original question, relevant schema, failed SQL, exact error, and a statement that execution returned no rows because it raised an error.

Retry behavior

- At most three corrections after the initial execution: four executions maximum.
- Attempted SQL strings are tracked; repeated SQL stops retries early.
- Corrections undergo the same fence cleanup and SELECT/WITH extraction.

Outcome	Meaning	Strategy
Rows	Success	Return immediately
Empty list	SQL ran; text condition may be strict	One grounded-value correction if possible
Exception	Syntax/schema/permission/other failure	General repair, up to three retries

Limitations: Error types do not yet have separate repair strategies. There is no independent read-only validator, so a read-only database account remains important.

9. Layer 6 - Answer Generation

Empty rows

Returns `No matching records were found.` without an LLM call.

Non-empty rows

The model receives the question, successful SQL, and all rows as JSON. It must state the direct answer first, format numbers, summarize patterns, avoid SQL/JSON discussion unless asked, and use only supplied facts.

Limitation: Result JSON currently has no row or character limit before entering the prompt.

Complete flow

```
Ensure metadata -> enrich -> build/reuse vectors -> embed question -> choose tables -> append FK targets -> choose columns -> compact schema -> generate SQL -> execute/correct -> summarize rows
```

Operational defaults

- Chat: `openai/gpt-oss-120b` on Groq, temperature 0.1.
- Embeddings: local Ollama `bge-m3`.
- Model logs can contain questions, schema identifiers, SQL, and results.